

Parciales viejos (MC)

Punto 1 - Semáforos



Resolver con **SEMÁFOROS** el siguiente problema. En una planta verificadora de vehículos existe un puesto de atención para atender a **20 vehículos** que deben hacer su verificación. En el puesto de atención hay **un empleado** que atiende a los vehículos de acuerdo con el orden de llegada, realiza la verificación y le entrega el comprobante; luego espera que el vehículo abandone el puesto para poder atender al siguiente.

Nota: existe la función *verificar(ID)* que simula que el empleado está verificando al vehículo ID; todos los procesos deben terminar.

Solución propuesta

```
sem espera_vehiculos = 0;
sem espera_salida = 0;
sem mutexCola = 1;
sem[] espera_atencion = ([20] 0);
cola<Integer> cola_atencion;

process vehiculo [Id: 1..20] {
    // Encolarse en la cola
    P(mutexCola);
    push(cola_atencion, Id);
    V(mutexCola);
    // Avisarle al empleado que llegó un vehículo
    V(espera_vehiculos);
    // Esperar a que haya sido atendido
    P(espera_atencion[Id]);
    // Sincronizarse junto al empleado, asegurarse de que salió
```

```

    V(espera_salida);
}

process empleado {
    for i:= 1 to N do
        // Esperar a que lleguen vehículos
        P(espera_vehiculos);
        // Si llegaron vehículos, es seguro sacar alguno de la cola
        P(mutexCola);
        id_auto = pop(cola_atencion);
        V(mutexCola);
        // Realizar la verificación del auto
        Verificar(id_auto);
        // Despertar al auto
        V(espera_atencion[id_auto]);
        // Esperar a que se haya retirado el vehículo para pasar al próximo
        P(espera_salida);
    }
}

```

Punto 2 - Monitores



Resolver con **MONITORES** el siguiente problema. En una oficina hay **un empleado** para atender solicitudes de ***N personas*** que vienen a realizar un trámite, de acuerdo con el orden de llegada. Cada persona espera a que el empleado atienda su trámite y le dé el resultado. Cuando el empleado está libre atiende el siguiente pedido pendiente, y si no hay ninguno lee un libro por 10 minutos.

Nota: existe la función *resolverTramite(solicitud)* que simula que el empleado está resolviendo el trámite de una persona; todos los procesos deben terminar.

Solución propuesta

```

process persona[Id: 1..N] {
    solicitud sol;
    tramite tram;

    // Encolarse junto con su solicitud, y esperar
    // a que el empleado le dé su trámite
    Oficina.llegar(Id, sol, tram);
}

process empleado {
    solicitud sol;
    bool llego_alg = false;
    int personas_pasaron = 0;
    int id_per;

    // Mientras no hayan pasado todas las personas...
    while (personas_pasaron < N)
        // Atender la solicitud de alguna persona, si es que llegó alguna
        Oficina.atender(id_per, sol, personas_pasaron, llego_alg);
        if (llego_alg)
            // Atender trámite y devolver resultado
            tramite tram = resolverTramite(sol);
            Oficina.entregar(id_per, tram, personas_pasaron);
        else
            // Simular que se va a leer un libro
            delay(10 min);
}

monitor oficina {
    tramite[] tramites_terminados = ([N]);
    cola<int, solicitud> = cola_solicitudes;
    cola<tramite> cola_tramite;
    cond[] espera_persona = ([N]);
    cond espera_confirmacion_recibio_tramite;
    int cont_personas_pasaron = 0;

```

```

procedure recibirTramite(Id: int in; sol: solicitud in, tra: tramite out) {
    cola_solicitudes.push(Id, sol);
    // Esperar a que el otro proceso nos procese la solicitud
    wait(espera_persona[Id]);
    // Recibimos el trámite
    tra = tramites_terminados[Id];
    // Incrementar el contador de personas que ya pasaron
    cont_personas_pasaron++;
    // Recien ahora avisarle al otro proceso que recibimos el trámite
    signal(espera_confirmacion_recibio_tramite);
}

procedure atender(Id: int out; sol: solicitud out; llevo_alg: bool out) {
    llevo_alg = not empty(cola_solicitudes);
    if (llevo_alg)
        Id, sol = cola_solicitudes.pop();
}

procedure entregar(Id: int in; tra: tramite in; pasaron: int out) {
    tramites_terminados[Id] = tra;
    signal(espera_persona[Id]);
    wait(espera_confirmacion_recibio_tramite);
    pasaron = cont_personas_pasaron;
}
}

```

Comentarios

- Me dijo el ayudante que, si se llegaba a utilizar un tipo de dato especial para una cuestión del ejercicio (en este caso sería la cola cuyos elementos son tuplas de **enteros** y **solicitudes**), se tendría que explicar bien como funciona o como está implementado, sin mostrar código obviamente.

Por ejemplo: en este caso el ayudante me recomendó que los elementos de la cola fueran registros que poseyeran dos valores, osea, si se hacía

cola_elem[0] se obtendría el entero, y si se hacía **cola_elem[1]** la solicitud. Yo creo que así como está queda claro pero bueno.

- También me dijo que quizás convenía hacer que el pasaje del trámite ocurriera por medio de un arreglo de tramites por Id, en vez de una variable permanente.
- Si uso un **while** en vez de un **for** es porque, así como tengo planteado el ejercicio, el for podría realizar una iteración sin que una solicitud fuese procesada (debido a que puede ser que no haya llegado ninguna persona y el empleado se quede leyendo). Para asegurarme que sí o sí hayan pasado las N personas, utilizo variables extra.

Parcial 14/05/2025

Programación Concurrente ATIC y Redictado - Parcial Práctico de MC - Primera Fecha - 14/05/2025

1. Resolver con **SEMÁFOROS** el siguiente problema. Se tiene un vector A de 1.000.000 de caracteres, del cual se debe obtener la cantidad de veces que aparecen los caracteres "F" y "C", utilizando **4 procesos Worker**. Al terminar todos los procesos deben imprimir la cantidad de veces que aparece cada una de esas dos letras (F y C). **Nota:** maximizar concurrencia; únicamente se pueden usar los 4 procesos *Worker*.
2. Resolver con **SEMÁFOROS** el siguiente problema. Existen **B barcos** areneros que deben descargar su contenido en una playa. Los barcos se descargan de a uno por vez, y de acuerdo con orden de llegada. Una vez que el barco llegó, espera a que le llegue su turno para comenzar a descargar su contenido, y luego se retira. **Nota:** sólo se pueden usar los procesos que representen a los barcos; cada barco descarga sólo una vez; suponga que existe la función *DESCARGAR()* que simula que el barco está descargando su contenido en la playa.
3. Resolver con **MONITORES** el siguiente problema. En una acopiadora de cereales hay **2 empleados**, uno para atender a los camiones de maíz y otro para los de girasol. Hay 30 camiones que llegan para descargar su carga (15 de maíz y 15 de girasol), cuando el camión llega espera hasta que el empleado correspondiente le avise que puede descargar el cereal. Cada empleado hace descargar los camiones que le corresponden de a uno a la vez y de acuerdo con orden de llegada. **Nota:** maximizar concurrencia; el camión sabe qué tipo de cereal lleva; todos los procesos deben terminar

Punto 1

Solución propuesta

```
char[] a = ... // Vector con 1.000.000 caracteres
int cant_car_C = 0;
int cant_car_F = 0;
```

```

int terminaron = 0;
sem mutex_contadores = 1;
sem espera_terminaron = 0;

// Para maximizar la concurrencia, lo mejor que
// se puede hacer es repartir al vector en zonas
// para que cada Worker procese en paralelo
// Esto evita la interferencia (ya que cada
// proceso va a tener su zona en la que solo él
// va a estar trabajando), y maximiza la concu-
// rrencia
process Worker [Id: 0..3] {
    int cant_car_C_local = 0;
    int cant_car_F_local = 0;
    int long_zona = (1.000.000 / 4)
    int pos_ini = long_zona * Id;

    for i:=1 to long_zona do
        char letra = a[i + pos_ini];
        if (letra == "C")
            cant_car_C_local += 1;
        else if (letra == "F")
            cant_car_F_local += 1;
    end;

    // No sé si haya que esperar a que todos los
    // procesos hayan agregado sus cantidades loca-
    // les al contador global, pero voy a hacerlo
    // de esa forma yo
    P(mutex_contadores);
    terminaron += 1;
    cant_car_C += cant_car_C_local;
    cant_car_F += cant_car_F_local;
    if (terminaron == 4)
        for i:=1 to 4 do
            V(espera_terminen);

```

```

V(mutex_contadores);
P(espera_terminen);

print(cant_car_C);
print(cant_car_F);
}

```

Comentarios

- A Guaymas le quedó bastante parecido :p

Punto 2

Solución incorrecta

```

bool descargando = false;
sem mutex = 1;
sem espera = 0;
int esperando = 0;

process barco [1..B] {
    P(mutex);
    if (descargando) then
        esperando++;
        V(mutex);
        P(espera);
    else
        V(mutex);
        descargando = false;
    Descargar();
    P(mutex);
    if (esperando > 0)
        esperando--;
        V(espera);
    else
        descargando = true;
}

```

```
V(mutex);  
}
```

Solución correcta

```
bool descargando = false;  
cola cola_ids;  
sem mutex = 1;  
sem espera[] = ([B] 0);  
  
process barco [Id: 1..B] {  
    P(mutex);  
    if (descargando) then  
        push(cola_ids, Id);  
        V(mutex);  
        P(espera[Id]);  
    else  
        descargando = false;  
        V(mutex);  
        Descargar();  
        P(mutex);  
        if (empty(cola_ids))  
            int aux_id = cola_ids.pop();  
            V(espera[aux_id]);  
        else  
            descargando = true;  
        V(mutex);  
}
```

Comentarios

- Siempre que se haya de mantener el orden de llegada, y solo hayan procesos de un tipo, se utiliza una cola. Hacerlo solo con variables contadoras y semáforos sirve en Monitores.

- Por otro lado, el libro dice que "Cuando un [proceso que tiene el batón] termina, si hay mas de un proceso esperando en el semáforo y uno se despierta, los demás se despiertan en orden de llegada." [[Fundamentals](#), p. 175]. Entonces, ¿por qué necesitaríamos una cola?
- No sé, pero en el parcial corregido está hecho como lo puse en la solución corregida, y Guaymas también lo tiene así, so shrug

Punto 3

Solución propuesta

```
process camion[Id: 1..30] {
    int num_empl; // El camión conoce el número de empleado que le toca
    puesto_atencion[num_empl].llegar();
    DescargarCereal()
    puesto_atencion[num_empl].salir();
}

process empleado[Id: 1..2] {
    for (int i = 0; i < 15; i++)
        puesto_atencion[Id].atender();
}

monitor puesto_atencion[1..2] {
    cola<Int> cola_ids;
    cond[] espera_camion = ([N]);
    cond espera_llegada;
    cond espera_descarga;
    bool usando = false;

    procedure llegar(id: int in) {
        cola_ids.push(id);
        signal(espera_llegada);
        wait(espera_camion[id]);
    }
}
```

```

procedure atender() {
    if (empty(cola_ids))
        wait(espera_llegada)
    aux_id = cola_ids.pop();
    signal(espera_camion[aux_id]);
    wait(espera_descarga);
}

procedure salir() {
    // Avisarle al empleado que terminó de descargar
    signal(espera_descarga);
}
}

```

- Si en el enunciado se nos dijese que solo habían procesos de un solo tipo, entonces la mejor solución era usar un "Passing the condition" sin colas, y con usar una variable contadora y un cond bastaba. Pero como hay otro tipo de proceso que se encarga de hacer pasar a los camiones, entonces si o si hay que usar una cola

Parcial 14/05/2025

Programación Concurrente ATIC y Redictado - Parcial Práctico de MC - Primera Fecha - 14/05/2025

- 1/ Resolver con **SEMÁFOROS** el siguiente problema. En un laboratorio de genética trabajan **20 empleados** que deben usar un pirosecuenciador de a uno a la vez, de acuerdo con el orden de llegada. **Nota:** sólo se pueden usar los procesos que representen a los empleados; cada empleado usa sólo una vez el pirosecuenciador; suponga que existe la función *USAR()* que simula el uso de este instrumento por parte del empleado.
2. Resolver con **SEMÁFOROS** el siguiente problema. Se tiene un vector A de 1.000.000 de números, del cual se debe obtener la suma de todos los valores utilizando **5 procesos Worker**. Al terminar todos los procesos deben imprimir el resultado final. **Nota:** maximizar concurrencia; únicamente se pueden usar los 5 procesos *Worker*.
3. Resolver con **MONITORES** el siguiente problema. En una oficina hay **un supervisor, un empleado y 50 personas** que solicitan por mail una operación. El empleado sólo atiende CONSULTAS, mientras que el supervisor sólo atiende TRÁMITES; cada uno atiende sus pedidos de acuerdo con el orden de llegada. Cada persona envía UNA solicitud que puede ser para un TRÁMITE o para una CONSULTA, y luego espera a que le envíen el resultado. **Nota:** maximizar concurrencia; la persona sabe de qué tipo es la consulta; el empleado y el supervisor NO deben terminar su ejecución.

Punto 1

Solución propuesta

```
bool libre = true;
cola cola_ids;
sem espera[] = ([20] 0);
sem mutex = 1;

process empleado[Id: 0..19] {
    P(mutex);
    if (not libre)
        push(Id, cola_ids);
        V(mutex);
        P(espera[Id]);
    else
        libre = false;
        V(mutex);

    USAR();

    P(mutex);
    if (!empty(cola_ids))
        int aux_id = pop(cola_ids);
        V(espera[aux_id]);
    else
        libre = true;
    V(mutex);
}
```

Punto 2

Solución propuesta

```
int[] A = ... // Datos, un millón de numeros
int suma = 0;
sem mutex = 1;
```

```

sem acceso = 0;
int termino = 0;

process Worker [Id: 0..4] {
    int suma_local = 0;
    int long_zona = 1.000.000 / 5;
    int pos_ini = long_zona * Id;

    for i:= 0 to long_zona do
        suma_local += A[i + pos_ini];
    end;

    P(mutex);
    termino += 1;
    suma += suma_local
    if (termino < 5)
        V(mutex);
        P(acceso);
    else
        for i:= 1 to 4 do
            V(acceso);
            V(mutex);

        print(suma);
    }
}

```

[ESTA EXCELENTE]

Punto 3

Solución propuesta

```

process persona [Id: 0..49] {
    int tipo_consulta = ...
    // La persona sabe de qué tipo es la consulta
}

```

```

    solicitud sol;
    resultado res;
    oficina[tipo_consulta].llegar(Id, sol, res);
}

process empleado [0..1] {
    solicitud sol_recibida;
    int id_persona;
    resultado res;

    while (True) {
        oficina[Id].atender(id_persona, sol_recibida);
        // Asumo que existe una función llamada procesarSolicitud(...)
        res = procesarSolicitud(sol_recibida);
        oficina[Id].responder(id_persona, res);
    }
}

monitor oficina [0..1] {
    cola<int, solicitud> cola_ids;
    cond espera_solicitud;
    cond[] espera_resultado = ([50]);
    resultado res_devolver;

    procedure llegar(id: int in; sol: solicitud in, res: resultado out) {
        cola_ids.push(id, sol);
        signal(espera_solicitud);
        wait(espera_resultado[Id]);
        res = res_devolver;
        signal(espera_persona);
    }

    procedure atender(id_persona: int out; sol_recibida: solicitud out) {
        if (empty(cola_ids))
            wait(espera_solicitud);
    }
}

```

```
        id_persona, sol_recibida = cola_ids.pop();
    }

    procedure responder(id_persona: int in; res: resultado in) {
        res_devolver = res;
        signal(espera_resultado[id_persona]);
        wait(espera_persona);
    }
}
```